Data Communications Project

Group Project Guidelines

Each group must select one topic of their choice, but the following rules should be observed:

- A group may consist of a **maximum of 3 students**.
- By the **end of the semester**, each group must present their work through both an **implementation/demo** and a **PowerPoint presentation**.
- The presentation should be **clear, well-structured, and easy for other classmates to understand**.
- **Every group member must take responsibility** for a specific part of the project and be familiar with the entire topic.
- **Scores will be based on teamwork**, contribution, and the overall quality of the project.
- **The dataset is flexible** — you can either create your **own** or use **publicly** available datasets.
- Uploading your code to **GitHub** can **earn incentive points**, but it is **not mandatory**.

NS-3

NS-3 (Network Simulator 3) is a discrete-event simulator designed for internet systems and wireless communication. Written in C++ with Python bindings, it supports technologies like TCP/IP, Wi-Fi, LTE, and 5G.

Why important: Provides high-fidelity simulation, scalability, and reproducible results for academic and industrial research.

Open access: Freely available, customizable, and supported by a large global community.

Installation (Linux/WSL/macOS):

https://www.nsnam.org/docs/release/3.45/installation/html/index.html?utm_source=chatgpt.com

Cooja

Cooja is the simulator of the Contiki and Contiki-NG operating systems, widely used in wireless sensor networks (WSNs) and IoT research. It can emulate real sensor motes, meaning the same code can run both in simulation and on hardware.

Why important: Enables testing of energy-efficient protocols and large-scale IoT systems before physical deployment.

Open access: Free to use, bridging the gap between theory and real devices.

Installation (Linux/macOS/VM on Windows)

https://docs.contiki-ng.org/en/master/doc/tutorials/Running-Contiki-NG-in-Cooja.html?utm_source=chatgpt.com

OMNeT++

OMNeT++ is a modular, component-based simulation framework with a strong visual interface. It is extended by frameworks like INET (internet protocols) and Veins (vehicular networks).

Why important: Combines flexibility, visualization, and extensibility for simulating wired, wireless, and IoT networks.

Open access: Free for academic use, with strong community support.

Installation (Linux/macOS)

https://omnetpp.org/download/?utm_source=chatgpt.com

Grafana

In the era of big data and distributed systems, monitoring and visualizing system performance has become essential. **Grafana** is one of the most popular **open-source observability platforms**, used for monitoring metrics, logs, and traces in real time. It provides a rich visualization interface, integrates with hundreds of data sources, and is widely adopted in DevOps, cloud computing, IoT, and research environments.

https://grafana.com/grafana/download

InfluxDB: Open-Source Time Series Database

Modern systems generate enormous volumes of time-stamped data from IoT devices, sensors, servers, and applications. Traditional relational databases struggle with storing and querying this data efficiently. InfluxDB, developed by InfluxData, is an open-source time series database (TSDB) designed specifically for handling high-speed writes and real-time queries. It is widely used for monitoring, analytics, IoT, and DevOps observability.

https://www.influxdata.com/downloads/

Eclipse Mosquitto: Open-Source MQTT Broker

In the Internet of Things (IoT) era, billions of devices communicate by sending small messages over unreliable or low-bandwidth networks. The MQTT (Message Queuing Telemetry Transport) protocol was designed for this purpose: lightweight, efficient, and reliable. Eclipse Mosquitto is a popular open-source MQTT broker that enables IoT devices, sensors, and applications to communicate using the publish/subscribe model.

https://mosquitto.org/download/

Apache Kafka: Distributed Event Streaming Platform

In today's digital world, systems must process and analyze **real-time data streams** at massive scale. Traditional messaging systems or databases often cannot handle this workload efficiently. **Apache Kafka**, originally developed at LinkedIn and now maintained by the Apache Software Foundation, is an **open-source distributed event streaming platform**. It is designed for **high-throughput, fault-tolerant, real-time data pipelines** and is widely used in industries such as finance, e-commerce, IoT, and cloud computing.

https://kafka.apache.org/downloads

Mininet: Open-Source Network Emulator

Modern networking research often requires testing protocols, topologies, and SDN (Software-Defined Networking) applications. Deploying on real hardware is expensive, complex, and time-consuming. Mininet is an open-source network emulator that allows researchers and students to create realistic virtual networks on a single machine. It uses lightweight virtualization to run real kernel, switch, and application code in a simulated environment, making it ideal for prototyping, education, and research in networking and SDN.

https://mininet.org/

ONOS: Open Network Operating System

With the rise of **Software-Defined Networking (SDN)**, networks are becoming programmable, flexible, and easier to manage. Instead of relying on closed, hardware-based solutions, SDN separates the **control plane** (decision-making) from the **data plane** (packet forwarding). **ONOS (Open Network Operating System)** is a **distributed, open-source SDN controller** developed by the Open Networking Foundation (ONF). It provides a **scalable, high-availability platform** for building next-generation carrier-grade and cloud networks.

https://opennetworking.org/onos/

---

1. Smart Home Energy Management with MQTT & Grafana

Objective

Design a full IoT-based home energy monitoring system that simulates smart appliances and sensors, transports data via MQTT, stores it in Influx DB, and visualizes trends in Grafana. Quantify latency, Packet Delivery Ratio (PDR), throughput, and Transport Layer Security (TLS) overhead, and (incentive) add AI/ML duty-cycling to reduce traffic/energy without sacrificing Quality of Service (QoS).

P0: Foundations

- Learn about the NS3 or Cooja simulator that you want to use in your scenario.
- Install NS-3 or Cooja; run a minimal publisher/subscriber sample.
- Spin up Mosquitto, InfluxDB, Grafana (Docker Compose is fine).

P1: Simulator Workload & Topology

- In NS-3/Cooja, create 10 sensors across rooms (kitchen, living, bedrooms).
- Traffic profiles (examples):
    - Power meter: 1 Hz (200–300 B payload)
    - Thermostat: 0.5 Hz + on-event updates (state changes)
    - Ambient sensors: 0.2–1 Hz
- Add network realism: 0–3% loss, 5–30 ms delay, 2–10 ms jitter.

P2: Messaging & Storage

- Configure Mosquitto (QoS 0 and QoS 1 experiments).
- Write subscriber/bridge to insert into InfluxDB (line protocol).
- Schema: measurement per metric (power, temp, humidity); tags: deviceId, room; fields: value(s).

P3: Dashboards & KPIs

- Grafana dashboards:
    - Overview: total house power, room breakdown, peak/off-peak.

    - Device drill-down: per-device time-series, on/off events.
- Instrument timing to compute latency segments:
    - L1: sensor→broker (publish to broker enqueue)
    - L2: broker→InfluxDB write

P4 — Experiments (Core Matrix)

- Scale: 10, 30, 60, 100 sensors
- QoS level: MQTT QoS
- Load shape: steady vs event-bursty

P5 — Security Study (Optional + Incentive)

- Enable TLS (self-signed CA; server cert on broker; clients verify).
- Re-run a subset of matrix (e.g., 30 & 100 sensors, steady + bursty).
- Report TLS overhead on latency/throughput and broker/subscriber CPU.

KPIs / QoS Outputs (must report):

- Latency (sensor → broker → dashboard): segment L1, L2, and E2E (report p50/p95/p99).
- Packet Delivery Ratio (PDR %) (at broker subscriber).
- Throughput (msgs/s) sustained under each scale/load.
- *(Nice-to-have)* Estimated energy proxy: msgs/s and bytes/s per sensor as a stand-in (or Cooja radio energy if used).

---

2. Comparative Study of MQTT, CoAP, and AMQP for IoT

Objective

Simulate MQTT, CoAP, and AMQP under identical IoT traffic in NS-3 or OMNeT++, study trade-offs under lossy, congested, and mobile scenarios, and (optionally) build an AI agent that recommends the best protocol per condition.

Phase 0 – Familiarization

- Install NS-3 or OMNeT++ and validate by running a small UDP/CoAP or MQTT example.
- Understand protocol basics:
    - MQTT: broker-based, reliable pub/sub.
    - CoAP: RESTful, UDP-based, very lightweight.
    - AMQP: broker-based, feature-rich, higher overhead.

Phase 1 – Baseline Implementation

- Implement scenarios for MQTT, CoAP, and AMQP with the same topology.

- Setup: broker/server + 10 IoT clients, periodic sensor traffic.
- Logging: per-packet timestamps, hop count, protocol headers, retries.

Phase 2 – Traffic Models

- Periodic: sensors send 64–256 B packets at 1 Hz.
- Bursty events: short 10–20 Hz bursts for 2–3 s.

Phase 3 – Network Scenarios

- Lossy channels: introduce 1–5% random packet loss.
- Congestion: high background traffic (non-IoT UDP/TCP).
- Mobility: some devices mobile at 1–5 m/s (e.g., drones, vehicles).

Phase 4 – Experiments

- Run each protocol under all scenarios and scales (10, 50, 100 devices).
- Collect QoS metrics for each run (see below).
- Repeat for ≥5 random seeds for statistical reliability.

Phase 5 – QoS Outputs (Mandatory)

- Delay (ms): average and p95 end-to-end latency.
- Jitter (ms): delay variation.
- Throughput (kbps): aggregated delivery rate.
- Energy consumption (mJ/bit): from radio model in NS-3/Cooja (if enabled).
- Scalability: success rate vs device count.

Phase 6 – Visualization

- Export logs → Influx DB → Grafana dashboards to show:
    - Delay/jitter over time by protocol.
    - Throughput vs device count.
    - Energy use trends under loss/mobility.

Phase 7 – Optional : AI Recommendation Engine (Incentive score)

- Goal: recommend the “best” protocol under given conditions (loss, congestion, mobility, device count).
- Method 1: Classification ML
    - Features: {device count, loss %, mobility speed, traffic type}.
    - Label: best protocol (lowest delay & energy, highest throughput).
    - Model: Random Forest or SVM.

- Method 2: Reinforcement Learning
    - Agent picks protocol for each scenario.
    - Reward = weighted QoS utility (e.g., U = −delay − energy + throughput).
    - Evaluate vs static choice baseline.

---

3. Routing & Networking – Protocol Comparison: AODV, RPL, DSR

Objective

Simulate AODV, RPL, and DSR in realistic IoT scenarios using NS-3 or Cooja, compare their performance under varying topologies and mobility, and (optional) extend with AI-based anomaly detection of routing misbehavior.

Phases & Detailed Tasks

Phase 0 – Familiarization

- Install and validate NS-3 or Cooja.
- Run a simple UDP client-server or ping scenario to understand simulation logging.

Phase 1 – Baseline Implementation

- Implement AODV, RPL, DSR separately in the chosen simulator.
- Build scripts/configs for repeatable runs.
- Logging: capture per-packet timestamps, sender/receiver, hop count, control messages.

Phase 2 – Topology Scenarios

Build four classes of IoT network topologies:

1. Static IoT network – 20–50 fixed nodes, grid/clustered.
2. Mobile IoT network – nodes as drones/vehicles (RandomWaypoint, Gauss-Markov).
3. Dense network – 100–200 nodes in 1 km² area (high contention).
4. Sparse network – 10–20 nodes, long distances, weak connectivity.

Phase 3 – Traffic Models

- Periodic sensor updates: UDP packets 64–256 B @ 1–2 Hz.
- Event-driven bursts: occasional 10–20 Hz bursts lasting 2–3 s.
- Identical traffic across all protocols for fairness.

Phase 4 – Experiments

- Run each routing protocol under each topology × traffic mix.
- Vary node density (10, 50, 100, 200).
- Vary mobility (static, low speed 1 m/s, high speed 10 m/s).

- Vary loss (0%, 2%, 5%).
- Collect results for at least 5 random seeds → compute averages & CIs.

Phase 5 – QoS Outputs (Mandatory)

Students must measure:

- Latency (ms): end-to-end average + p95.
- Jitter (ms): variance in delay for periodic flows.
- Packet Delivery Ratio (PDR %).
- Routing Overhead (%): ratio of control packets to data packets.

Phase 6 – Visualization

- Export logs (CSV/JSON) into Influx DB.
- Build Grafana dashboards to show:
    - Latency/jitter trends per protocol.
    - PDR over time under different loads.
    - Routing overhead comparisons.

Phase 7 – Optional: AI Anomaly Detection (Incentive score)

- Goal: detect abnormal routing behaviors such as:
    - Blackhole attack (malicious node drops packets).
    - Excessive control overhead (misconfigured node floods HELLO messages).
- Dataset:
    - Features: per-node control msg count, avg hop count, packet loss rate.
    - Labels: normal vs anomaly.
- Models: Random Forest, Isolation Forest, Autoencoder (unsupervised).
- Output: detection accuracy (≥0.85 F1 for good score).

---

4. Vehicular Ad-Hoc Network (VANET) Safety Communications + AI Congestion Control

Objective

Model a realistic **urban VANET** in **NS-3**, compare **AODV vs OLSR** for multi-hop safety messaging under mobility, and (optional) add **unsupervised AI** to detect broadcast storms or misconfigurations.

Tools (open)

- **NS-3** (802.11p/WaveNetDevice) + FlowMonitor

- Mobility: **SUMO** (import via TraCI/trace) or NS-3 mobility helpers
- Analysis/AI: Python (pandas, numpy, matplotlib/seaborn, scikit-learn)

Scenario (high level)

- **Area:** 1–2 km² urban grid (4–16 intersections), optional RSUs at major junctions
- **Vehicles:** 30 → 120; speeds 30–60 km/h; stop-and-go at lights
- **Radio:** 5.9 GHz 802.11p, tx power 10–23 dBm, MCS 6/12 Mbps
- **Traffic:** 10 Hz safety beacons (200–400 B) + event alerts (500–800 B bursts)

Phases & Detailed Tasks

P0. Foundations

- Install NS-3; run Wave/802.11p examples; learn FlowMonitor & PCAP logging.

P1. Baseline VANET

- **Mobility:** import SUMO routes or configure Grid/Waypoint with lights/turns.
- **PHY/MAC:** 802.11p config (channel, tx power, data rate, ED threshold).
- **App layer:** periodic beacons + event alerts on simulated hard brake.
- **Logging:** per-packet tx/rx time, sender/receiver IDs, RSSI/SINR, hop count.

P2. Routing Protocols

- Enable **AODV** and **OLSR** (separate runs, identical seeds/topologies).
- Validate route formation and stability.

P3. Experiments

- **Density sweep:** 30 / 60 / 120 vehicles.
- **Speed sweep:** 30 vs 60 km/h.
- **Environment:** LOS vs NLOS (buildings module if used); hidden terminals.
- **Background load:** optional infotainment UDP (e.g., 256 kbps/car).
- **High-mobility case:** platoon split/merge, intersection congestion.

P4. Analysis & Plots

- Build **delay/jitter/PDR vs distance** (0–100, 100–200, 200–300 m bins) and **vs density** for AODV vs OLSR.
- CDFs of event-message delay (tail behavior ≥100 ms).
- Map/heatmap of PDR across the grid (optional).

P5. Optional + Incentive: AI Congestion/Anomaly Detection

- **Objective:** detect **broadcast storms** or **misconfigured nodes** in real time.

- **Features (per node/time-window):** channel busy ratio, tx rate, retries, queue length, neighbor count, duplicate frames.
- **Methods:** Isolation Forest / One-Class SVM / Autoencoder.
- **Evaluation:** precision/recall/F1 using seeded anomalies (e.g., a node stuck at 50 Hz beacons).

---

5. Blockchain-enabled IoT Communication over MQTT (Security vs Overhead)

Objective:

Design and evaluate an MQTT → blockchain data pipeline that provides integrity, provenance, and auditability for IoT streams, and quantify the performance overhead vs a no-chain baseline.

Tools (all open/free)

- Network & traffic: OMNeT++ (INET/IoT modules) to simulate IoT nodes, loss/jitter, bursts.
- Messaging: Mosquitto (MQTT broker).
- Blockchain (permissioned preferred):
    - Hyperledger Fabric (RAFT ordering, channels, private data) or
    - Hyperledger Besu / GoQuorum (Ethereum PoA/IBFT).
    - (Dev-only rapid tests: Hardhat/Ganache.)
- Off-chain storage: InfluxDB (time-series), optional IPFS for raw blobs.
- Analytics: Python (pandas, numpy, matplotlib/seaborn/scikit-learn) + Grafana dashboards.

0) Foundations

- Install/validate OMNeT++ with a tiny publisher scenario.
- Spin up Mosquitto and a local Fabric test-network *or* Besu/Quorum PoA devnet.
- Run a “hello world” tx on-chain (store a dummy hash), and a basic MQTT pub/sub.

1) Baseline MQTT & Workload

- **Device classes (examples):**
    - *Traffic sensor:* 200 B @ 2 Hz
    - *Air quality:* 300 B @ 0.5 Hz

    - *Vibration/industrial:* 300 B @ 20 Hz (bursty)
    - *Alerts:* 400–600 B on events (rare bursts)
- **Scales:** 50 → 200 → 500 simulated devices across classes.
- **Network realism:** in OMNeT++, add 0–5% packet loss, 5–30 ms delay, 2–10 ms jitter.
- **Baseline path:** MQTT → InfluxDB (no blockchain). Log end-to-end latency & PDR.

2) Gateway/Adapter + Security

- **Message schema** (JSON/Avro): {deviceId, ts_device, seq, payload, sig/HMAC}.
- **Device identity:** simulate per-device keys (Ed25519/ECDSA). Devices sign payload.
- **Gateway:**
    - Verify signature/HMAC; reject tampered/late (>Δt) messages.
    - **Batching policies:**
        - Size-based N ∈ {50, 200, 500}
        - Time-based Δ ∈ {250, 500, 1000} ms
    - Build **Merkle tree** per batch; store root on-chain (Anchoring mode).
    - Optionally **Full-record mode** (compact fields on-chain).
    - Persist raw to **InfluxDB** (and/or IPFS) with Merkle proof.

3) Blockchain Network & Config

- Fabric: tune batchTimeout (0.5/1/2 s), block size (1–4 MB); define channel; simple chaincode to write (root, ts, count, appId).
- Besu/Quorum (PoA/IBFT): block time 1/2/5 s; validators 4/7; Solidity contract to append (root, metadata).
- Backpressure: if chain is slow, gateway queues; measure queue depth & retries.

4) Experiments (matrix)

- **Scale:** 50/200/500 devices × mixed rates.
- **Mode:** *No-chain* vs *Anchoring* vs *Full-record*.
- Consensus params: (see above).
- **Batching:** N or Δ sweep.

- **Stress:** broker overload; inject 2–5% extra loss; CPU-cap orderers/validators.

5) Analytics & Dashboards

- **Latency breakdown:**
    - L1: device→MQTT broker
    - L2: broker→gateway ingest
    - L3: gateway→on-chain confirmation (receipt/block commit)
    - E2E: device publish → confirmation (and → InfluxDB write)
- **Throughput:** ingest msgs/s and on-chain TPS; queue lag; failure rate.
- **Grafana panels:** ingest rate, on-chain confirmation p50/p95/p99, backlog, failures, CPU/RAM (optional).

6) (Optional + Incentive) ML Anomaly Detection

- **Goal:** detect suspicious patterns (duplicate hashes, out-of-order timestamps, frequent tx failures, abnormal submit intervals).
- **Features:** tx interval stats, dup ratio, fail streaks, mempool depth (if available), gateway queue depth.
- **Models:** Isolation Forest or Autoencoder (unsupervised).
- **Labeling:** seed anomalies (scripted duplicates/out-of-order/failure spikes).
- **Metric:** precision/recall/F1; alert on Grafana.

---

6. 5G-enabled IoT Communication (4G LTE vs 5G NR)

Goal

Build a comparative **NS-3** testbed for **LTE (LENA)** and **5G NR (ns-3-nr)** that carries **mixed IoT traffic** (sensors + video), runs **load & radio parameter sweeps**, evaluates **mobility & handover**, and reports **QoS**.

Scope & Assumptions

- **Open tools only**: NS-3 (LENA + ns-3-nr), Python (pandas, numpy, matplotlib/seaborn), optional Grafana for monitoring.
- **Traffic classes**: low-rate sensors (UDP), video telemetry (UDP or TCP).
- **Cells**: multi-cell layout to force handovers.

Architecture & Scenario Design

- **Area**: 1–2 km² urban grid; 3–7 eNB/gNB sites (optionally tri-sector).

- **UEs**: 50–200 sensor UEs + 5–20 video UEs (scalability runs).
- **Radio configs**
  - **LTE**: 10/20 MHz @ 2.6 GHz, 2×2 MIMO, Proportional Fair (PF) scheduler.
  - **NR**: 40/100 MHz @ 3.5 GHz, numerology $\mu \in \{0,1\}$ (15/30 kHz SCS), 2×2 or 4×4 MIMO, PF or Round-Robin (RR) scheduler.
- **Mobility**: pedestrian (1.5 m/s) and vehicular (10–15 m/s) paths crossing cell borders.
- **Traffic models**
  - **Sensors**: UDP CBR, 64–256 B @ 1–10 Hz + bursty alert mode (short spikes to 10–20 Hz).
  - **Video**: VBR traces or OnOff approximations @ 1–6 Mbps (720p/1080p profiles).
- **Logging**: FlowMonitor + custom traces (timestamps, cell IDs, RSRP/RSRQ/CQI, HO events).

Detailed Tasks

1) Testbed Bring-up (LTE & NR)

- Instantiate **LTE EPC** & eNBs; attach UEs; enable CQI reporting.
- Instantiate **NR gNBs**; configure bandwidth (40/100 MHz), **numerology** $\mu=0$ and $\mu=1$; choose scheduler PF or RR.
- Implement identical app-layer generators for LTE and NR (same port/rates/sizes).

2) Load & Radio Parameter Sweeps

- **Load**: run matrices with {50, 100, 200} sensor UEs × {5, 10, 20} video UEs.
- **Bandwidth**: LTE {10, 20} MHz; NR {40, 100} MHz.
- **MCS**: static low/med/high vs adaptive (CQI-driven).
- **Numerology** (NR): $\mu=0$ vs $\mu=1$ to study slot duration vs latency.
- **Schedulers**: PF vs RR (NR); LTE baseline PF.
- **Link budget stress**: vary tx power; add shadowing to create cell-edge users.

3) Mobility & Handover

- Configure A3-event handover with **hysteresis** (1–3 dB) and **Time-to-Trigger** (80–160 ms).
- Run pedestrian vs vehicular mobility; create overlapping coverage to induce HOs.
- **Measure** interruption time, HO failures, ping-pong rate, RLF recoveries.

4) QoS Comparison Under Mixed Traffic

- For each matrix point, record **sensor** (latency, jitter, PDR) and **video** (goodput, loss %, stall proxies).
- Compare LTE vs NR; analyze **where** NR wins most (e.g., high load, $\mu=1$, wide BW).

7. Cloud-based Smart City IoT Data Platform

Goal: Build an end-to-end smart-city data platform that ingests simulated sensor streams (traffic, air quality, weather) into a cloud streaming pipeline, processes them in near-real time, and visualizes results. Quantify latency, throughput, and scalability up to 1000+ devices, and demonstrate fault tolerance and multi-tenant (city zones) isolation. Optional AI improves autoscaling and anomaly detection.

Architecture (high level)

- Edge/City devices (simulated): NS-3 or OMNeT++ publishers (traffic loops, PM2.5/NO_2, temp/humidity, rain).
- Gateway (optional): MQTT broker (Mosquitto) → Kafka bridge (or publish directly to Kafka REST/producer).
- Streaming layer: Kafka topics (per domain & per zone) → Spark Structured Streaming jobs (ETL, aggregation, anomaly detection).
- Storage: InfluxDB (time-series)
- Visualization/ops: Grafana dashboards (real-time KPIs), alerting (thresholds).
- Orchestration: Docker Compose (simple) or Kubernetes/Kind (for autoscaling experiments).

Phase 1 — City & Workload Modeling (Simulation)

Domains & zones: Define 3 domains × 3–5 zones:

- Traffic: vehicle count, avg speed, occupancy, incidents.
- Air quality: PM2.5/PM10/NO_2/CO, sensor health.
- Weather: temp, humidity, pressure, rainfall, wind.

Simulation (choose one):

- NS-3: UDP/TCP publishers with loss/jitter models per zone; optional SUMO to drive traffic features.
- OMNeT++ (INET/IoT): application generators per sensor type; inject bursty events (incidents, storms).

Scalability matrix: 100, 300, 600, 1000+ devices; message sizes 100–600 B; rates 0.2–5 Hz; loss 0–3%.

Multi-tenant tagging: include city_zone, device_type, device_id in every message (JSON or Avro).

Phase 2 — Streaming Ingest & Schema

Kafka setup: topics per domain+zone, e.g., traffic.z1, air.z3, partitions scaled with device count (e.g., 6–24).

Serialization: Avro or JSON; include timestamp (device + gateway), seqno, QoS flags.

Phase 3 — Real-time Processing (Spark Structured Streaming)

8. Pipelines:
    - Traffic: rolling 1-min/5-min averages (speed, density), incident detection (sudden speed drop).
    - Air: AQI computation per zone; calibrate sensors with simple linear correction.
    - Weather: micro-climate aggregates; rainfall burst detection.
9. State & watermarks: late data handling (1–2 min), exactly-once semantics with checkpointing.

Phase 4 — Storage & Dashboards

Influx DB schema: measurement per domain; tags: zone, device_type, device_id; fields: numeric values.

Grafana:

- Overview: per-domain KPIs (mean/95p latency, ingest rate, backlog, error rate).

Phase 5 — Experiments (Quantitative Study)

Latency pipeline: measure sensor→Kafka, Kafka→Spark in, Spark proc time, Spark→Influx, and end-to-end.

Throughput & backpressure: ramp devices; record stable msgs/s before lag grows.

Scalability: increase Kafka partitions & Spark executors; show effect on lag and E2E latency.

Fault tolerance: broker outage (n seconds), Spark restart, one partition hot-spot; quantify data loss/duplication & recovery time.

KPIs & QoS Outputs (must report)

- Latency (ms): per segment and end-to-end (sensor→Grafana); p50/p95/p99.
- Throughput (msgs/s): sustained & peak; Kafka consumer lag (records & time).
- Scalability: max device count with p95 E2E latency ≤ X ms (define X, e.g., 2000 ms).
- Availability/Recovery: time to recover stream after failure; % data preserved (exactly-once vs at-least-once).
- Data quality: late/duplicate rate (%), out-of-order handling effectiveness

---

8. IoT Traffic Management with Cloud-based SDN

Objective: Use Software Defined Network (**SDN)** to steer mixed IoT traffic through a virtual WAN toward **cloud resources**, and quantify the gains of **dynamic, controller-driven routing** versus **static routing**.

Phases & Detailed Tasks (no weekly schedule)

P0 — Foundations

Bring up **Mininet** vanilla topo (linear, tree), verify ping/iperf.

Install **ONOS** or **OpenDaylight**; connect **OVS** switches (OpenFlow 1.3).

Push a “hello world” flow (host-to-host) from the controller.

P1 — Baseline (Static Routing)

Static paths: use OVS flow entries or Linux routes (no controller decisions).

Generate mixed traffic; collect **baseline KPIs** (see KPIs).

P2 — SDN Control (Dynamic Routing)

Topology discovery: Link Discovery app; verify host learning.

Intent-based routing (ONOS) / Flow Objectives:

- Install host-to-host **Intents** to Cloud DC A.
- **Reactive vs Proactive**: compare packet-in driven vs pre-installed flows.

Load-aware steering: implement a **link utilization probe** (sFlow-RT or custom) and steer new flows to the **least-loaded** path or to **Cloud DC B** when A nears saturation.

QoS queues/meters: classify traffic into classes (Telemetry/Control/Bulk).

- Configure **OVS QoS** (htb) with **priority queues**; apply with **meters** or DSCP matching.

Fast reroute / failure: drop a core link; verify **controller reconvergence** and flow re-installation.

P3 — Experiments (Comparative Study)

Design a matrix and run ≥5 seeds per point for CIs:

- **Scale:** 10, 40, **100+** IoT hosts
- **Pathing:** Static vs SDN **(reactive / proactive / load-aware)**
- **Impairments:** add netem delay 10/30/50 ms and loss 0/0.5/1% on one trunk
- **Congestion:** add bulk uploads (TCP) from gateways (0/2/5 flows)
- **Failover:** single core link failure during run (t = 120 s)

P4 — Optional : Forecast-aware Steering (Incentive score)

- Train a simple **predictor** (ARIMA/Prophet or LSTM/Random Forest) on recent link utilization to **pre-emptively** steer upcoming flows (or switch DC selection) to keep latency low for telemetry/control.
- Compare to reactive load-aware policy.

P5 — Monitoring & Dashboards

- Export sFlow/Prometheus stats (per link/port/queue).
- **Grafana** panels:
  - End-to-end latency (ping/active probes), **flow setup time** (packet-in → first data pkt)
  - Link **utilization** & **queue occupancy**
  - Controller stats (packet-in rate, flow installs/s), reconvergence time

KPIs / QoS Outputs (must report)

- **Latency (ms):** p50/p95 end-to-end for telemetry & control flows
- **Flow setup time (ms):** controller **reactive** install time (packet-in → usable path)
- **Bandwidth utilization (%):** per trunk/queue (time-avg and peaks)
- **Loss % / PDR:** for telemetry under congestion/failure
- **Failover time (ms):** traffic recovery after link failure (first success after blackout)
- *(Nice-to-have)* Controller overhead: pkt-in/s, flow-mods/s

---